

Programação Orientada a Objetos

Herança e Composição em Java

Professor: Me. Sergio Souza Novak
Faculdade de Engenharia de Software

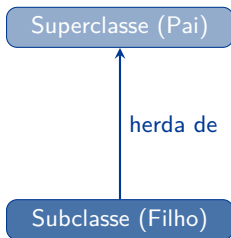


- 1 Introdução
- 2 Conceitos de Herança
- 3 Exemplo: Hierarquia Animal
- 4 Exemplo Prático: Equipamentos Médicos
- 5 Modificadores de Acesso e Herança
- 6 A Palavra-chave `super`
- 7 Conceitos de Composição
- 8 Implementação de Composição
- 9 Conclusão: Herança vs Composição



O que é Herança?

- **Conceito:** É um mecanismo que permite que uma classe (subclasse) herde atributos e métodos de outra classe (superclasse).
- **Reutilização:** Promove a reutilização de código, evitando duplicações.
- **Relação "É um":** Define uma relação onde a subclasse é uma especialização da superclasse.
- **Exemplo:** Um *Cachorro* é um *Animal*.



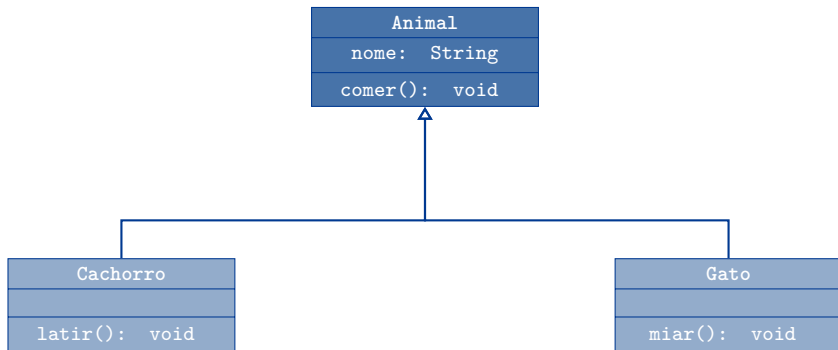
Como a Herança ocorre?

- Em Java, utilizamos a palavra-chave `extends`.
- Uma classe só pode estender **uma única** classe (Herança Simples).
- A subclasse ganha acesso a todos os membros `public` e `protected` da superclasse.

Sintaxe

```
public class Subclasse extends Superclasse { ... }
```





Exemplo de Herança: Superclasse

```
1 // Classe Base (Superclasse)
2 public class Animal {
3     String nome;
4
5     public void comer() {
6         System.out.println(nome + " esta comendo...");
7     }
8 }
```



Exemplo de Herança: Subclasse Cachorro

```
1 // Subclasse
2 public class Cachorro extends Animal {
3     public void latir() {
4         System.out.println(nome + " diz: Au Au!");
5     }
6 }
```



Exemplo de Herança: Subclasse Gato

```
1 // Subclasse
2 public class Gato extends Animal {
3     public void miar() {
4         System.out.println(nome + " diz: Miau!");
5     }
6 }
```



Exemplo de Herança: Uso

```
1 public class Main {  
2     public static void main(String[] args) {  
3         Cachorro c1 = new Cachorro();  
4         c1.nome = "Rex";  
5         c1.comer(); // Herdado de Animal  
6         c1.latir(); // Proprio de Cachorro  
7  
8         Gato g1 = new Gato();  
9         g1.nome = "Luna";  
10        g1.comer(); // Herdado de Animal  
11        g1.miar();  // Proprio de Gato  
12    }  
13 }
```



Superclasse

Classe Animal

- Define características comuns:
 - Atributo: nome
 - Método: comer()
- Serve como base para outras classes

Subclasses

Cachorro e Gato

- Herdam de Animal usando extends
- Reutilizam:
 - nome
 - comer()
- **Especialização:**
 - Cachorro → latir()
 - Gato → miar()

- **Conceito:** Base tecnológica para diagnósticos por imagem via radiação.
- **Funcionamento:** Produção de feixes de elétrons em uma ampola de vácuo.
- **Atributos Comuns:**
 - Modelo e Fabricante
 - Tensão de Operação (kV)



- **Especialização:** Realiza múltiplos cortes (slices) do corpo.
- **Funcionalidade:** Reconstrução de imagens em 3D.
- **Uso:** Diagnóstico detalhado de órgãos internos e vasos.

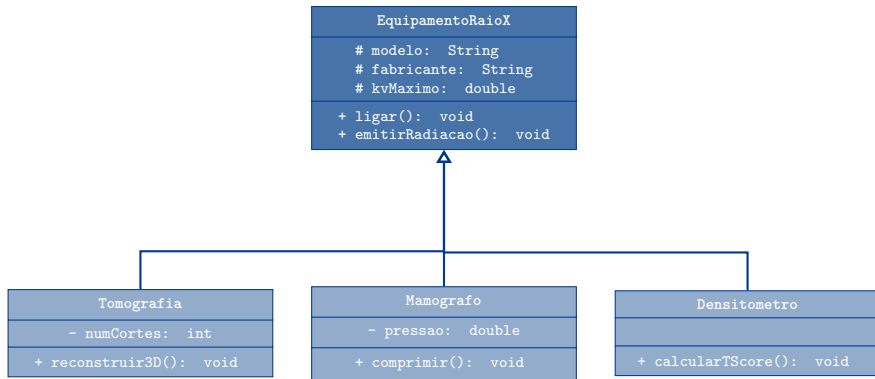


- **Especialização:** Exames de tecidos mamários.
- **Funcionalidade:** Compressão mecânica para melhor nitidez.
- **Uso:** Prevenção e diagnóstico de câncer de mama.



- **Especialização:** Medição da densidade mineral óssea.
- **Funcionalidade:** Cálculo do T-Score.
- **Uso:** Diagnóstico de Osteoporose e Osteopenia.





Exemplo Complexo: Superclasse

```
1 public class EquipamentoRaioX {  
2     protected String modelo;  
3     protected String fabricante;  
4     protected double kvMaximo; // Kilo-voltagem  
5  
6     public void ligar() {  
7         System.out.println("Equipamento " + modelo + " ligado.");  
8     }  
9  
10    public void emitirRadiacao() {  
11        System.out.println("Emitindo raios-X em " + kvMaximo + "kV  
12        ...");  
13    }
```



Subclasse: Tomografia

```
1 public class Tomografia extends EquipamentoRaioX {  
2     private int numeroCortes;  
3  
4     public void reconstruir3D() {  
5         System.out.println("Reconstruindo imagem de " +  
6             numeroCortes + " cortes.");  
7     }  
8 }
```



Subclasse: Mamógrafo

```
1 public class Mamografo extends EquipamentoRaioX {  
2     private double pressaoCompressor;  
3  
4     public void comprimir() {  
5         System.out.println("Comprimindo com " +  
6                             pressaoCompressor + "kgf.");  
7     }  
8 }
```



Subclasse: Densitômetro Ósseo

```
1 public class Densitometro extends EquipamentoRaioX {  
2     public void calcularTScore() {  
3         System.out.println("Calculando densidade ossea...");  
4     }  
5 }
```



Exemplo Complexo: Uso

```
1 public class TesteClinica {  
2     public static void main(String[] args) {  
3         Tomografia t1 = new Tomografia();  
4         t1.modelo = "Aquilion";  
5         t1.ligar(); // Herdado  
6         t1.reconstruir3D(); // Especifico  
7  
8         Mamografo m1 = new Mamografo();  
9         m1.modelo = "Senographe";  
10        m1.emitirRadiacao(); // Herdado  
11        m1.comprimir(); // Especifico  
12    }  
13 }
```



Herança e Reutilização

Como classes compartilham comportamento



Modificador `private` e Herança

- **Acesso:** Apenas dentro da própria classe.
- **Herança:**
 - Não é acessível diretamente na subclasse
 - Ainda existe no objeto (é herdado, mas oculto)
- **Exemplo:**

```
1 class Animal {  
2     private String nome;  
3 }  
4  
5 class Cachorro extends Animal {  
6     void teste() {  
7         // nome = "Rex"; ERRO  
8     }  
9 }
```

Conclusão

Use getters/setters para acessar atributos privados.

Modificador `private` e Herança

- **Acesso:** Apenas dentro da própria classe.
- **Herança:**
 - Não é acessível diretamente na subclasse
 - Ainda existe no objeto (é herdado, mas oculto)
- **Exemplo:**

```
1 class Animal {  
2     private String nome;  
3 }  
4  
5 class Cachorro extends Animal {  
6     void teste() {  
7         // nome = "Rex";  ERRO  
8     }  
9 }
```

Conclusão

Use getters/setters para acessar atributos privados.

Modificador public e Herança

- **Acesso:** Qualquer classe em qualquer lugar.
- **Herança:**
 - Totalmente acessível na subclasse
 - Pode ser usado livremente
- **Exemplo:**

```
1 class Animal {  
2     public String nome;  
3 }  
4  
5 class Cachorro extends Animal {  
6     void teste() {  
7         nome = "Rex"; // permitido  
8     }  
9 }
```

Conclusão

Fácil acesso, mas expõe demais os dados.

Modificador default (package-private)

- Sem modificador declarado
- Acesso:
 - Apenas dentro do mesmo pacote
- Herança:
 - Acessível se a subclasse estiver no mesmo pacote
 - Não acessível em pacotes diferentes
- Exemplo:

```
1 class Animal {  
2     String nome; // default  
3 }  
4  
5 class Cachorro extends Animal {  
6     void teste() {  
7         nome = "Rex"; // mesmo pacote  
8     }  
9 }
```

Importante

Não funciona bem para herança entre pacotes.



A Palavra-chave super

O que é?

Referência direta à **superclasse** imediata

Para que serve?

- Chamar o construtor da superclasse
- Acessar métodos sobrescritos
- Acessar atributos herdados

Regra de Ouro

`super()` deve ser a **primeira instrução** no construtor

Superclasse



Cachorro

Subclasse

```
Cachorro() {  
    super();  
}
```

- Quando criamos um objeto da subclasse, o Java precisa inicializar a superclasse primeiro.
- Se a superclasse não tiver um construtor padrão (vazio), a subclasse **deve** chamar explicitamente um construtor da superclasse.

Sintaxe

```
super(parâmetros);
```



Exemplo: super() no Construtor

```
1 class EquipamentoRaioX {
2     String modelo;
3     EquipamentoRaioX(String modelo) {
4         this.modelo = modelo;
5     }
6 }
7
8 class Tomografia extends EquipamentoRaioX {
9     int cortes;
10    Tomografia(String modelo, int cortes) {
11        super(modelo); // Chama o pai primeiro!
12        this.cortes = cortes;
13    }
14 }
```



- Usado quando a subclasse **sobrescreve** um método do pai, mas ainda deseja aproveitar o comportamento original.
- Permite adicionar comportamento novo sem descartar o antigo.

Sintaxe

```
super.nomeDoMetodo();
```



Exemplo: super.metodo()

```
1 class EquipamentoRaioX {
2     void emitirRadiacao() {
3         System.out.println("Radiacao base emitida.");
4     }
5 }
6
7 class Tomografia extends EquipamentoRaioX {
8     @Override
9     void emitirRadiacao() {
10         super.emitirRadiacao(); // Comportamento base
11         System.out.println("Iniciando varredura 3D...");
12     }
13 }
```



O que é Composição?

- **Conceito:** É um tipo de relacionamento onde uma classe "tem um"(has-a) objeto de outra classe como atributo.
- **Flexibilidade:** Permite mudar o comportamento em tempo de execução ao trocar as instâncias dos objetos internos.
- **Acoplamento:** Geralmente resulta em um acoplamento mais fraco do que a herança.

Exemplo

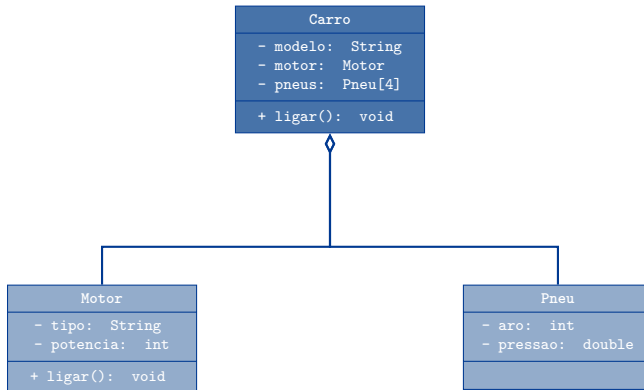
Um *Computador* não "é um" *Processador*, ele **tem um** *Processador*.



Herança vs. Composição

Característica	Herança (Is-a)	Composição (Has-a)
<i>Relação</i>	Especialização ("É um")	Agregação ("Tem um")
<i>Visibilidade</i>	Quebra o encapsulamento (acesso ao protected)	Mantém o encapsulamento
<i>Flexibilidade</i>	Estática (tempo de compilação)	Dinâmica (tempo de execução)
<i>Princípio</i>	Reuso por extensão	Reuso por delegação

UML: Exemplo de Composição



Composição: O ciclo de vida das partes (*Motor*, *Pneu*) está ligado ao todo (*Carro*).

- Consiste em declarar atributos cujos tipos são outras classes.
- **Delegação:** A classe principal delega tarefas para os objetos que ela compõe.

Sintaxe

```
public class Todo {  
    private Parte objetoParte;  
}
```



```
1 public class Motor {  
2     private String tipo;  
3     private int potencia;  
4  
5     public Motor(String tipo, int potencia) {  
6         this.tipo = tipo;  
7         this.potencia = potencia;  
8     }  
9  
10    public void ligar() {  
11        System.out.println("Motor " + tipo + " de " +  
12                               potencia + "cv ligado!");  
13    }  
14 }
```

```
1 public class Carro {  
2     private String modelo;  
3     private Motor motor; // Composicao (Has-a)  
4  
5     public Carro(String modelo, Motor motor) {  
6         this.modelo = modelo;  
7         this.motor = motor;  
8     }  
9  
10    public void darPartida() {  
11        System.out.print(modelo + " dando a partida: ");  
12        motor.ligar(); // Delegacao  
13    }  
14 }
```

```
1 public class MainComposicao {  
2     public static void main(String[] args) {  
3         // Criamos a "parte"  
4         Motor m1 = new Motor("V8", 450);  
5  
6         // Criamos o "todo" passando a parte  
7         Carro c1 = new Carro("Mustang", m1);  
8  
9         c1.darPartida();  
10        // Mustang dando a partida: Motor V8 de 450cv ligado!  
11    }  
12 }
```

- **Flexibilidade:** Podemos passar diferentes motores para o mesmo carro (Motor Eletrico, Motor Flex) sem alterar a classe Carro.
- **Manutenibilidade:** Alterações em Motor não afetam o funcionamento interno de Carro, desde que a interface seja mantida.
- **Simplicidade:** Objetos menores são mais fáceis de entender, testar e manter.



- **Relacionamento "É um"(Is-a):** Quando a subclasse é claramente uma especialização da superclasse.
- **Polimorfismo:** Quando você precisa tratar diferentes objetos de forma genérica através do tipo da superclasse.
- **Estabilidade:** Quando a hierarquia de classes é estável e as mudanças no topo devem refletir na base.

Dica

Se você não consegue dizer com confiança que "A é um B", não use herança.



Quando usar Composição?

- **Relacionamento "Tem um"(Has-a):** Quando um objeto é composto por outros componentes.
- **Flexibilidade:** Quando você precisa trocar partes do sistema sem afetar a estrutura principal.
- **Encapsulamento:** Quando você quer esconder os detalhes internos dos componentes.
- **Baixo Acoplamento:** Para evitar as fragilidades de uma hierarquia de herança profunda.

Dica

Prefira Composição sempre que o reuso de código não exigir uma relação de parentesco estrita.

